

MMD-GAN on MNIST

The first attempt in full detail, the final solution explained part by part,
and the reason behind every change

Contents

1	What problem are we solving?	2
2	A 60-second primer on how an MMD-GAN trains	2
3	The first attempt, in full detail	2
3.1	Configuration	3
3.2	The critic: a plain feature extractor	3
3.3	The kernel and the unbiased MMD ²	3
3.4	The witness and the (one-sided) gradient penalty	3
3.5	The activation penalty (the central new idea)	4
3.6	Optimisers, EMA generator, and learning-rate decay	4
3.7	The training loop, step by step	5
3.8	Honest scoring (KID) and saving	5
3.9	Summary: what each modification was meant to do	5
3.10	What actually happened, and why	5
4	The final solution, explained part by part	6
4.1	The one change that fixes everything	6
4.2	Configuration	6
4.3	The generator (unchanged DCGAN)	7
4.4	The autoencoder critic (the key part)	7
4.5	The adaptive RBF kernel and unbiased MMD ²	8
4.6	The loss helpers	8
4.7	EMA generator, optimisers, schedulers	9
4.8	The honest score (frozen LeNet KID)	9
4.9	The training loop, step by step	9
5	Why each modification, first attempt → final	10
6	Side-by-side comparison	10
7	Why the final design cannot collapse to black	10
8	How to run and what to watch	10

1 What problem are we solving?

We train a Generative Adversarial Network (GAN) to produce MNIST digits, using **Maximum Mean Discrepancy (MMD)** as the adversarial objective. Two earlier runs failed in opposite ways:

- One run let the critic’s internal numbers grow without bound; sample quality wandered and degraded late in training.
- The **first attempt** in this document tried to fix that by forcing those numbers to a fixed size. It *over-corrected*: the model never learned at all — the quality score (KID) was frozen around 3.0 for the whole run and the generated images turned almost completely black.
- The **final solution** rebuilds the critic as an *autoencoder*, which removes the failure entirely.

This document explains the first attempt with every detail, then explains every part of the final code, and gives the reasoning behind each modification.

2 A 60-second primer on how an MMD-GAN trains

You only need four ideas to follow everything below.

1. The generator G . Takes random noise z and outputs an image. Goal: produce images that look like real MNIST digits.

2. The critic (feature extractor) D . Maps an image to a vector of numbers called *features* (or a *code*). It is *not* a yes/no classifier — it just produces a representation.

3. MMD — a distance between two *sets of points*. Given the feature vectors of a batch of real images and a batch of fake images, $\widehat{\text{MMD}}_u^2$ measures how far apart the two clouds of points are. It is 0 when the two clouds have the same distribution and positive otherwise. It is built from a *kernel* $k(a, b)$ that measures similarity between two feature vectors (the RBF kernel $\exp(-\|a - b\|^2/2\sigma^2)$ is the usual choice).

4. The adversarial game.

- The **critic** adjusts its features to make the real and fake clouds *far apart* (maximise MMD) — it tries to expose differences.
- The **generator** adjusts its images to make the two clouds *overlap* (minimise MMD) — it tries to hide differences.

At equilibrium the generator’s images are indistinguishable from real ones in the critic’s feature space. A regulariser is always needed to stop the critic from “cheating” by blowing up its features to make MMD artificially huge.

***Intuition.** Think of the critic as a judge who invents a measuring stick, and the generator as a forger. The judge wants a measuring stick that exposes fakes; the forger wants forgeries that pass the stick. The whole difficulty is keeping the judge’s measuring stick meaningful — if the judge is allowed to use a silly stick, the forger can pass it with garbage.*

3 The first attempt, in full detail

The first attempt kept the standard structure (plain critic, kernel-based MMD, gradient penalty) and added an **activation penalty** whose job was to stop the critic’s features from growing too large. Below is every component.

3.1 Configuration

```
@dataclass
class Config:
    img_size: 28; channels: 1; batch_size: 128
    z_dim: 100                                # noise dimension
    feat_dim: 64                              # number of critic features (MMD is computed on these)
    g_filters: 64; d_filters: 32              # generator / critic width

    kernel: 'rq'                              # rational-quadratic mixture kernel

    lambda_gp: 1.0                           # weight of the gradient penalty
    lambda_act: 1.0                           # weight of the NEW activation penalty
    act_target: 1.0                           # target size for the features

    lr_G: 1e-4; lr_D: 2e-4                   # learning rates (critic slightly faster: "TTUR")
    beta1: 0.5; beta2: 0.9
    n_critic: 1                              # critic updates per generator update
    n_iters: 50000
    lr_decay_start: 25000                     # start decaying learning rate halfway
    log_every: 200; sample_every: 1000
```

The three lines that matter for this attempt are `lambda_act`, `act_target` (the activation penalty), and `kernel='rq'`.

3.2 The critic: a plain feature extractor

The critic is a small convolutional network that turns a 28×28 image into a 64-dimensional feature vector via a final linear “head”. It uses LayerNorm (not BatchNorm, which is incompatible with the gradient penalty) and outputs the features directly, with no constraint on how large they can be. **This freedom is the seed of the problem.**

3.3 The kernel and the unbiased MMD²

MMD is computed with a mixture of rational-quadratic kernels $k(a, b) = \sum_{\alpha} (1 + \|a - b\|^2 / 2\alpha)^{-\alpha}$ over several α values (a multiscale kernel is more robust than a single bandwidth). The unbiased estimator drops the diagonal of the kernel matrices:

$$\widehat{\text{MMD}}_u^2(u, v) = \frac{1}{m(m-1)} \sum_{i \neq j} k(u_i, u_j) + \frac{1}{n(n-1)} \sum_{i \neq j} k(v_i, v_j) - \frac{2}{mn} \sum_{i, j} k(u_i, v_j).$$

3.4 The witness and the (one-sided) gradient penalty

The gradient penalty keeps the critic approximately 1-Lipschitz (its output cannot change too fast as the input changes), which is what makes MMD a well-behaved objective.

```
def witness(x, real_feat, fake_feat, critic, kind):
    # fhat(x): how much more "real-like" than "fake-like" the point x is
    f_x = critic(x)                                # gradient flows here
    k_real = kernel_matrix(real_feat, f_x, kind).mean(0)
    k_fake = kernel_matrix(fake_feat, f_x, kind).mean(0)
    return k_real - k_fake
```

```
def gradient_penalty(critic, real, fake, kind):
    bs = real.size(0)
    alpha = torch.rand(bs, 1, 1, 1, device=real.device)
    interp = (alpha*real + (1-alpha)*fake).requires_grad_(True)  # points between real and fake
    with torch.no_grad():
        real_feat = critic(real); fake_feat = critic(fake)
    w = witness(interp, real_feat, fake_feat, critic, kind)
    grads = torch.autograd.grad(w.sum(), interp, create_graph=True)[0]
    grad_norm = grads.view(bs, -1).norm(2, dim=1)
    return F.relu(grad_norm - 1).pow(2).mean()  # ONE-SIDED: only penalise slopes > 1
```

Why we changed it. The original code used a two-sided penalty $(\|\nabla\| - 1)^2$, which forces the slope to be *exactly* 1 everywhere. We switched to the one-sided form $\text{relu}(\|\nabla\| - 1)^2$, which only punishes slopes *above* 1. This is gentler and a common stability tweak.

3.5 The activation penalty (the central new idea)

This is the heart of the first attempt.

```
def activation_penalty(f_real, f_fake, target):
    ms_real = f_real.pow(2).mean()  # average squared feature value (real)
    ms_fake = f_fake.pow(2).mean()  # average squared feature value (fake)
    pen = F.relu(ms_real - target) + F.relu(ms_fake - target)  # punish only when above target
    return pen, 0.5*(ms_real + ms_fake).item()
```

It computes the mean squared value of the critic’s features and penalises it only when it rises above `act_target=1`. Formally, with $\bar{s} = \frac{1}{Bd} \sum f_{ij}^2$,

$$\text{act_pen} = \text{relu}(\bar{s}_{\text{real}} - 1) + \text{relu}(\bar{s}_{\text{fake}} - 1).$$

Why we added it. In the earlier (broken) run the critic features grew to enormous values (mean squared value in the thousands). When features are huge, every pair of points is “far”, so the kernel returns nearly the same number for all pairs and MMD loses its gradient. The activation penalty was meant to pin the feature *size* near 1 so the kernel stays in its sensitive range.

3.6 Optimisers, EMA generator, and learning-rate decay

```
G_ema = deepcopy(G).eval(); EMA_DECAY = 0.999  # smoothed copy of the generator for sampling
opt_G = Adam(G.parameters(), lr=1e-4, betas=(0.5,0.9))
opt_D = Adam(D.parameters(), lr=2e-4, betas=(0.5,0.9))

def _lr_factor(step):  # 1.0, then linear decay to 0 after halfway
    if step < cfg.lr_decay_start: return 1.0
    span = cfg.n_iters - cfg.lr_decay_start
    return max(0.0, 1.0 - (step - cfg.lr_decay_start)/span)
sched_G = LambdaLR(opt_G, _lr_factor); sched_D = LambdaLR(opt_D, _lr_factor)
```

- **EMA generator** (`G_ema`): a running average of the generator’s weights, used only for sampling/scoring. Averaged weights give smoother, more stable images than the raw, still-moving generator.

Why: reduces sample noise from step to step.

- **Learning-rate decay**: the learning rate is held constant, then linearly lowered to 0 over the second half. *Why*: smaller late-training steps reduce the wandering we saw in the earlier run.

3.7 The training loop, step by step

```
for it in range(1, n_iters+1):
    # --- 1) Critic update (repeated n_critic times) ---
    for _ in range(n_critic):
        real = next_real_batch()
        fake = G(randn(...)).detach()           # generator frozen here
        f_real, f_fake = D(real), D(fake)       # features
        mmd2 = mmd2_unbiased(f_real, f_fake, KERNEL)
        gp = gradient_penalty(D, real, fake, KERNEL)
        act_pen, _ = activation_penalty(f_real, f_fake, act_target)
        d_loss = -mmd2 + lambda_gp*gp + lambda_act*act_pen  # maximise MMD, obey both
    penalties
    opt_D.zero_grad(); d_loss.backward()
    clip_grad_norm_(D.parameters(), 1.0); opt_D.step()

    # --- 2) Generator update ---
    fake = G(randn(...))
    f_real = D(real).detach(); f_fake = D(fake)
    g_loss = mmd2_unbiased(f_real, f_fake, KERNEL) # minimise MMD
    opt_G.zero_grad(); g_loss.backward(); opt_G.step()

    sched_G.step(); sched_D.step()               # learning-rate decay
    ema_update(G_ema, G)                        # smoothed generator

    # --- 3) Periodic logging / sampling / scoring ---
    if time_to_sample:
        kid = kid_estimate()                     # quality score (lower = better)
        if kid < best_kid: save("checkpoints/best.pt") # keep the best model ever seen
```

Reading it as a story: (1) the critic looks at a real batch and a (frozen) fake batch and adjusts its features to push the two apart, while obeying the gradient penalty and the activation penalty; (2) the generator then adjusts its images to pull the two feature clouds back together; (3) every 1000 steps we sample from the smoothed generator, score it with KID, and keep the best-scoring model on disk.

3.8 Honest scoring (KID) and saving

KID is computed on the features of a *separate, frozen* LeNet digit classifier — never on the critic’s own (adversarially-trained) features, which would be a biased ruler. We also added saving of every sample grid, the learning curves, a text log, `history.json`, and a **best-by-KID checkpoint**, so a late dip in quality can never lose the best model.

3.9 Summary: what each modification was meant to do

3.10 What actually happened, and why

Result: the model never learned. The feature size `feat^2` sat right around 0.7–1.05 the entire run (so the penalty “worked” at its literal job), but the quality score KID was *stuck* at 3.0–3.4 for all

Modification	Intended effect
Activation penalty (<code>lambda_act=1, act_target=1</code>)	stop features exploding; keep the kernel sensitive
One-sided gradient penalty	gentler Lipschitz constraint, more stable
Learning-rate decay (second half)	calm the late-training wandering
Best-by-KID checkpoint	never lose the best model to late drift
Save all figures / logs / history	full record of every run

50,000 iterations, and the samples collapsed to near-black images. (An earlier run without this penalty had reached KID ≈ 0.82 .)

Why. Nothing in the first attempt forces the features to *mean* anything. The activation penalty controls only their *size*. So the critic was free to separate real from fake using a content-free signal (for example, overall brightness), and it did: it kept MMD high on a trivial statistic. The generator, trying to minimise that statistic, found the cheapest solution — output a nearly uniform black image. High MMD reported by the critic, terrible images, frozen KID. *We fixed a symptom (size) but not the disease (uninformative features).*

Intuition. *We forced the judge’s measuring stick to be a fixed length, but never required it to measure anything useful. The judge picked a useless stick (“how dark is the image?”), and the forger passed it perfectly by submitting a black rectangle.*

4 The final solution, explained part by part

4.1 The one change that fixes everything

Make the critic an **autoencoder**: it still produces a code (used for MMD), *but it must also reconstruct the input image from that code*. A reconstruction loss forces the code to contain real image content. Now “match the codes” genuinely means “match the digits,” and a black image cannot pass, because a code that ignores the digit cannot rebuild it.

Intuition. *We force the judge’s measuring stick to be reversible: from the measurement alone you must be able to redraw the original image. A stick that only measures brightness fails this test, so the judge is forced to measure real digit structure — and now the forger can only pass by drawing real digits.*

4.2 Configuration

```
@dataclass
class Config:
    img_size: 28; channels: 1; batch_size: 128
    z_dim: 100
    feat_dim: 32          # size of the autoencoder CODE (the MMD feature)
    g_filters: 64          # generator / decoder width
    d_filters: 32          # encoder width
    lambda_ae: 8.0         # weight of the reconstruction loss (keeps codes informative)
    lr_G: 1e-4; lr_D: 1e-4
    beta1: 0.5; beta2: 0.9
    n_critic: 5            # train the critic 5x harder than the generator
    n_iters: 50000; lr_decay_start: 40000
    log_every: 200; sample_every: 1000
```

Gone: `lambda_gp`, `lambda_act`, `act_target`, the kernel choice. New: `lambda_ae` (reconstruction weight). `n_critic` is now 5.

4.3 The generator (unchanged DCGAN)

```
class Generator(nn.Module):
    # z(100) -> 7x7 -> 14x14 -> 28x28 image in [-1,1]
    self.net = Sequential(
        ConvTranspose2d(z_dim, ngf*4, 7,1,0), BatchNorm2d, ReLU, # -> (ngf*4, 7, 7)
        ConvTranspose2d(ngf*4, ngf*2, 4,2,1), BatchNorm2d, ReLU, # -> (ngf*2,14,14)
        ConvTranspose2d(ngf*2, ngf, 4,2,1), BatchNorm2d, ReLU, # -> (ngf, 28,28)
        Conv2d(ngf, 1, 3,1,1), Tanh()) # -> (1, 28,28)
```

A standard DCGAN generator: it upsamples a 100-dim noise vector through transposed convolutions to a 28×28 image, ending in `Tanh` so pixels lie in $[-1, 1]$ (matching the normalised MNIST data).

4.4 The autoencoder critic (the key part)

```
class AECritic(nn.Module):
    def __init__(self, channels=1, ndf=32, ngf=64, code_dim=32):
        # ENCODER: image -> code. Each layer is spectral-normalised (Lipschitz control).
        self.encoder = Sequential(
            spectral_norm(Conv2d(channels, ndf, 4,2,1)), LeakyReLU(0.2), # 28 -> 14
            spectral_norm(Conv2d(ndf, ndf*2, 4,2,1)), LeakyReLU(0.2), # 14 -> 7
            spectral_norm(Conv2d(ndf*2, ndf*4, 4,2,1)), LeakyReLU(0.2), # 7 -> 3
            Flatten(),
            spectral_norm(Linear(ndf*4*3*3, code_dim))) # -> code (32)
        # DECODER: code -> image (mirrors the generator)
        self.dec_fc = Sequential(Linear(code_dim, ngf*4*7*7),
                                BatchNorm1d(ngf*4*7*7), ReLU())
        self.decoder = Sequential(
            ConvTranspose2d(ngf*4, ngf*2, 4,2,1), BatchNorm2d, ReLU, # 7 -> 14
            ConvTranspose2d(ngf*2, ngf, 4,2,1), BatchNorm2d, ReLU, # 14 -> 28
            Conv2d(ngf, channels, 3,1,1), Tanh()) # -> image

    def encode(self, x): return self.encoder(x)
    def decode(self, c): return self.decoder(self.dec_fc(c).view(-1, ngf*4, 7, 7))
    def forward(self, x):
        code = self.encode(x)
        return code, self.decode(code) # returns BOTH the code and the reconstruction
```

Part by part:

- **Encoder** — three convolutions shrink the image $28 \rightarrow 14 \rightarrow 7 \rightarrow 3$, then a linear layer produces the 32-dim *code*. This code is the feature fed to MMD.
- **spectral_norm on every encoder layer** — divides each layer's weight matrix by its largest singular value, so the layer cannot amplify its input by more than ~ 1 . This keeps the whole encoder Lipschitz (bounded sensitivity), which is what the gradient penalty used to provide — but automatically and per layer, with no extra loss term and no clamping of the feature scale.
- **Decoder** — mirrors the generator: a linear layer reshapes the code to 7×7 , then transposed convolutions grow it back to 28×28 , ending in `Tanh`. It tries to reconstruct the original image from the code.

- forward returns *both* the code (for MMD) and the reconstruction (for the reconstruction loss). encode is a shortcut for when only the code is needed.

4.5 The adaptive RBF kernel and unbiased MMD²

```

RBF_MULTS = (0.25, 0.5, 1.0, 2.0, 4.0)          # bandwidth multipliers around the median

def _sqdist(x, y):                                # all pairwise squared distances
    xx = (x*x).sum(1, keepdim=True); yy = (y*y).sum(1, keepdim=True).t()
    return (xx + yy - 2*(x @ y.t())).clamp_(min=0)

def _median_scale(d2):                            # median distance -> the kernel's base
    bandwidth
    n = d2.size(0); mask = ~torch.eye(n, dtype=bool, device=d2.device)
    return d2[mask].median().detach().clamp_min(1e-8)

def _rbf_mix(d2, base):                          # mixture of RBFs at several bandwidths
    return sum(torch.exp(-d2 / (c*base)) for c in RBF_MULTS)

def mmd2_rbf_adaptive(f_x, f_y):
    base = _median_scale(_sqdist(cat([f_x,f_y]), cat([f_x,f_y]))) # bandwidth from THIS
    batch
    k_xx = _rbf_mix(_sqdist(f_x,f_x), base)
    k_yy = _rbf_mix(_sqdist(f_y,f_y), base)
    k_xy = _rbf_mix(_sqdist(f_x,f_y), base)
    # unbiased estimator: drop the diagonals of k_xx and k_yy, then combine
    return diag_dropped(k_xx) + diag_dropped(k_yy) - 2*mean(k_xy)

```

Part by part:

- `_sqdist` — the squared distance between every pair of feature vectors.
- `_median_scale` — the **median-distance heuristic**: it sets the kernel's bandwidth σ_0^2 to the median pairwise distance of the current batch. The `.detach()` means this scale is treated as a constant (no gradient through it).
- `_rbf_mix` — evaluates several RBFs at bandwidths $\{0.25, 0.5, 1, 2, 4\} \times \sigma_0^2$ and sums them, giving a multiscale kernel.
- `mmd2_rbf_adaptive` — builds the three kernel matrices and combines them into the unbiased MMD².

Why adaptive bandwidth? Spectral norm keeps the codes *small* (here $\|\text{code}\| \approx 0.2$). A fixed bandwidth like $\sigma = 1$ would be far too large for such small distances — the kernel would return ≈ 1 for every pair and MMD would carry no gradient. The median heuristic sets the bandwidth to match whatever scale the codes actually take, so the kernel is always in its sensitive range. (Verified: it separates correctly at code scales 0.2, 1, and 5.)

4.6 The loss helpers

```

EPS = 1e-6
def mmd2_codes(code_x, code_y):                  # clamped MMD^2 on the codes
    return F.relu(mmd2_rbf_adaptive(code_x, code_y))
def mmd_sqrt(code_x, code_y):                    # sqrt(MMD^2): better-scaled gradients
    return torch.sqrt(mmd2_codes(code_x, code_y) + EPS)
def recon_loss(rec, target):                     # how well the decoder rebuilt the image
    return F.mse_loss(rec, target)

```


relu guards against tiny negative values from the unbiased estimator; the square root is the objective the reference implementation uses because it gives gradients of a more uniform scale than raw MMD²; recon_loss is plain mean-squared reconstruction error.

4.7 EMA generator, optimisers, schedulers

Identical in spirit to the first attempt: an EMA copy of the generator for sampling, Adam optimisers (now *equal* learning rates 10^{-4}), and a gentle linear LR decay over the tail (lr_decay_start=40000). The EMA update now also copies BatchNorm *buffers*, so the averaged generator samples correctly in eval mode.

4.8 The honest score (frozen LeNet KID)

```
def kid_estimate(n_samples=2000, kind='rq'):
    # MMD between real and generated images, measured on a SEPARATE frozen LeNet's features
    real_f = lenet.embed(real_images)
    fake_f = lenet.embed(G_ema(noise))
    return mmd2_unbiased(real_f, fake_f, kind).item()    # lower = better
```

This is kept unchanged from before. The point is that the ruler (a frozen MNIST classifier) is *independent* of the critic, so it is an honest measure of sample quality. Lower is better.

4.9 The training loop, step by step

```
for it in range(1, n_iters+1):
    # --- 1) Critic (encoder + decoder), n_critic = 5 times ---
    for _ in range(n_critic):
        real = next_real_batch()
        fake = G(randn(...)).detach()                # generator frozen here
        code_r, rec_r = D(real)                       # code + reconstruction (real)
        code_f, rec_f = D(fake)                       # code + reconstruction (fake)
        mmd2 = mmd2_codes(code_r, code_f)             # separation between codes
        recon = recon_loss(rec_r, real) + recon_loss(rec_f, fake) # reconstruction quality
        d_loss = -torch.sqrt(mmd2 + EPS) + lambda_ae * recon
        opt_D.zero_grad(); d_loss.backward(); opt_D.step()

    # --- 2) Generator (codes only) ---
    fake = G(randn(...))
    code_r = D.encode(real).detach()                 # real codes are fixed targets
    code_f = D.encode(fake)                          # gradient flows back into G
    g_loss = mmd_sqrt(code_r, code_f)
    opt_G.zero_grad(); g_loss.backward(); opt_G.step()

    sched_G.step(); sched_D.step(); ema_update(G_ema, G)

    # --- 3) Periodic sampling, KID, best checkpoint, figures/log saved to disk ---
```

As a story:

1. **Critic phase (5x).** Encode real and fake images to codes *and* reconstruct them. The loss `-sqrt(mmd2) + lambda_ae*recon` pulls the critic in two directions at once: *push the codes apart* (good discriminator) *and reconstruct well* (informative codes). The reconstruction term is the safety rope that keeps the discrimination honest.

2. **Generator phase.** Encode a fresh fake batch; the gradient of `sqrt(mmd2)` flows back through the encoder into the generator, pushing the fake codes to overlap the real codes. Because the codes now encode digit structure, overlapping them means the fake images look like real digits.
3. **Bookkeeping.** Decay LR, update the EMA generator, and every 1000 steps sample from `G_ema`, compute KID, and keep `checkpoints/best.pt`; all figures and logs are written to `outputs/`.

5 Why each modification, first attempt $\rightarrow$ final

6 Side-by-side comparison

7 Why the final design cannot collapse to black

The failure needed one ingredient: a critic feature that the generator could satisfy with a blank image. In the final design that ingredient is gone. A black image encodes to some code; the decoder will reconstruct *black* from that code; but real digits encode to codes that reconstruct *digits*. So a black image's code is necessarily far from the real-digit codes, MMD between them is large, and the generator is penalised for going blank. The reconstruction loss is what ties the code to the image, and spectral norm keeps the whole thing numerically well-behaved without the scale-clamping that broke the first attempt.

8 How to run and what to watch

- Run top to bottom. It needs a GPU and downloads MNIST automatically.
- `n_critic=5` makes each iteration $\sim 5\times$ heavier on the critic. For a quick look, set `n_critic` to 2–3 and `n_iters` to ~ 15 –20k.
- In the log, watch `recon` (should fall fast early) and `code||` (should stay bounded). If reconstruction stays high, raise `lambda_ae`; if MMD^2 stays near zero, lower it.
- `checkpoints/best.pt` always holds the lowest-KID model. Reload with `G_ema.load_state_dict(torch.load('checkpoints/best.pt')['G_ema'])`.
- The code was smoke-tested end to end but not run to full convergence, so treat the hyperparameters as a strong starting point.

Change	Reason (full explanation)
Plain critic → autoencoder critic	The plain critic had no reason to make its features meaningful, so it separated real/fake on a content-free statistic and the generator collapsed to black. Forcing the critic to also <i>reconstruct</i> the image makes the code carry real content, so matching codes = matching digits. This is the change that actually fixes the failure.
Activation penalty + gradient penalty → spectral normalisation	The activation penalty controlled feature <i>size</i> (wrong lever) and the gradient penalty was fiddly. Spectral norm gives the needed Lipschitz constraint directly inside the layers, automatically, without clamping the feature scale or adding a tuning knob.
Fixed/mixture kernel → adaptive (median-heuristic) bandwidth	Spectral norm makes codes small; a hand-set bandwidth would sit in the kernel’s flat zone and kill the gradient. The median heuristic sets the bandwidth from the data each step, so the kernel is always responsive.
Raw $\text{MMD}^2 \rightarrow \sqrt{\text{MMD}^2}$	The square root gives gradients of a more uniform magnitude across training, as in the reference implementation; more stable optimisation.
<code>n_critic</code> 1 → 5	MMD-GANs need a well-trained critic each step; five critic updates per generator update gives a sharper, more reliable signal (at $\sim 5\times$ the critic cost).
Equal learning rates	With spectral norm + autoencoder the critic no longer needs the faster “TTUR” rate; equal rates are simpler and stable.
Kept: EMA, LR decay, KID, best checkpoint, file saving	These were already good ideas and are orthogonal to the fix, so they carry over unchanged.

Component	First attempt	Final
Critic	plain conv $\rightarrow$ feature vector	autoencoder: encoder $\rightarrow$ code + decoder $\rightarrow$ reconstruction
Lipschitz control	one-sided gradient penalty	spectral norm on the encoder (GP removed)
Keeps features sensible	activation penalty on feature <i>size</i>	reconstruction loss on feature <i>content</i>
Kernel	fixed-shape RQ mixture	RBF mixture with median-heuristic bandwidth
MMD objective	raw MMD ²	$\sqrt{\text{MMD}^2}$
n_critic	1	5
Learning rates	$10^{-4}/2 \cdot 10^{-4}$ (TTUR)	$10^{-4}/10^{-4}$
EMA / KID / best.pt / saving	yes	yes (kept)
Outcome	KID frozen ≈ 3.0 ; images collapse to black	informative codes; collapse-to-black removed